TEACHING SUMMARY

Computer Science – Form Two

Pages 215–223

These notes are kept very simple for beginner students. Use the Easy explanation after each point when teaching.

Main topics: Arrays, strings, file handling, debugging, errors and debugging tools.

Page 215 – Arrays and Strings

1. Array

An array is a group of values stored under one name.

Easy explanation: An array is like a row of boxes. Each box keeps one value. Each box has a number called an index.

```
Index: 0 1 2 3 4
Value: 1 2 3 4 5
```

Remember: In C, array counting starts from 0.

Example

```
#include
int main() {
int numbers[5] = {1,2,3,4,5};
for (int i=0; i<5; i++) {
printf("%d ", numbers[i]);
}
return 0;
}
```

Easy explanation: The loop moves from the first value to the last value.

2. Print in reverse

```
for (int i=4; i>=0; i--) {
printf("%d ", numbers[i]);
}
```

Easy explanation: Start from the last position and move backwards.

```
Output: 5 4 3 2 1
```

3. String

A string is text, such as a word or a name.

Easy explanation: COMPUTER is a string because it is a group of characters.

```
char word[20] = "COMPUTER";
```

4. Counting characters

A C string ends with a special character called \0. We can count characters until we reach it.

```
int count = 0;
while (word[count] != '\0') {
count++;
}
```

Easy explanation: The program counts one character at a time until it reaches the end.

Page 216 – Simple String Ideas

Full name

scanf with %s normally reads one word. A full name may contain spaces.

Easy explanation: John is one word. John Peter contains two words.

printf() and puts()

printf() displays formatted output. puts() mainly displays a string.

```
printf("Age = %d", age);
puts("Hello students");
```

Easy explanation: printf() is more flexible. puts() is simple for displaying text.

Another example

```
int age = 16;
printf("My age is %d", age);
puts("Welcome to Computer Science");
```

Easy explanation: The first line prints a value with text. puts() simply prints text.

Pages 217–219 – File Handling

1. What is file handling?

File handling means using a C program to open, read, write and close a file.

Easy explanation: A file is like a notebook. We can put information in it and use it later.

2. Main file functions

Function	Simple meaning
fopen()	Open a file
fprintf()	Write data to a file
fscanf()	Read data from a file
fclose()	Close a file

3. File modes

Mode	Simple meaning
r	Read
w	Write / replace old content
a	Add at the end
r+	Read and write
w+	Write and read
a+	Add and read

4. Writing to a file

```
FILE *file;
file = fopen("students.txt", "w");
fprintf(file, "John\n");
fprintf(file, "Peter\n");
fclose(file);
```

Easy explanation: Open the file, write the names, then close the file.

5. Reading from a file

```
FILE *file;
char word[50];
file = fopen("data.txt", "r");
fscanf(file, "%s", word);
printf("%s", word);
fclose(file);
```

Easy explanation: Open the file, read the information, display it, then close the file.

Easy memory: r = read | w = write/replace | a = add at the end

Page 220 – Debugging

1. What is debugging?

Debugging is finding and fixing errors in a program.

Easy explanation: When a program does not work correctly, we look for the problem and correct it.

2. Compilation / syntax error

This happens when the code breaks the rules of C.

```
printf("Hello")
```

Easy explanation: The semicolon is missing.

```
printf("Hello");
```

3. Logical error

The program runs but gives the wrong answer.

```
int a=5;
int b=2;
int result=a-b;
```

Easy explanation: If we want addition, use a+b. The program can run, but the answer is wrong.

4. Runtime error

A runtime error happens while the program is running.

```
int a=10;
int b=0;
int result=a/b;
```

Easy explanation: The program tries to divide by zero.

Page 221 – Three Important Errors

Error	Easy meaning	Example
Syntax	Code is written wrongly.	Missing ;
Logical	Program runs but answer is wrong.	Use - instead of +
Runtime	Problem happens while running.	Divide by zero

Easy memory: Syntax = writing mistake. Logic = wrong calculation. Runtime = problem while running.

Example of a logical error

```
int marks=50;
int bonus=20;
int total=marks-bonus;
```

Easy explanation: If the bonus should be added, use + instead of -.

Preventing division by zero

```
if (b != 0) {
result = a / b;
} else {
printf("Cannot divide by zero");
}
```

Easy explanation: Check the number before dividing.

Page 222 – Simple Debugging Methods

1. Read the error message

Read the compiler message carefully. It gives a clue about the problem.

> Easy explanation: If it says a semicolon is expected, check for a missing ;.

2. Use printf() to check values

```
int a=5;
int b=2;
int result=a+b;
printf("a=%d\n",a);
printf("b=%d\n",b);
printf("result=%d\n",result);
```

> Easy explanation: This lets us see the values inside the program.

3. Fix the program step by step

```
Write code
↓
Compile
↓
Find error
↓
Fix error
↓
Compile again
```

> Easy explanation: Find the problem, fix it, then test again.

4. Debugger

A debugger is a tool that helps a programmer find problems in a program.

> Easy explanation: It helps us stop the program and see what is happening.

Page 223 – Debugging Tools and Exercise 4.10

1. Step through code

Run the program one line at a time.

Easy explanation: This helps you see what each line is doing.

2. Breakpoint

A breakpoint tells the debugger to stop at a chosen point.

Easy explanation: Think of a breakpoint as a STOP sign inside the program.

3. Watch variables

The debugger can show the current values of variables.

```
a = 5
b = 2
result = 7
```

Easy explanation: If a value is not correct, look for the mistake.

Exercise 4.10 – Question 1

If printf() is missing a semicolon, add it.

```
printf("Hello, World!\n");
```

Easy explanation: This is a syntax error.

Question 2

If the program should add numbers but uses minus, change - to +.

```
sum = num1 + num2;
```

Easy explanation: This is a logical error.

Question 3

If the program divides by zero, check the second number before dividing.

```
if (num2 != 0) {
result = num1 / num2;
} else {
printf("Cannot divide by zero.");
}
```

Easy explanation: This prevents the runtime problem.

Final memory: Array = group of values. File = stores information. Debugging = finding and fixing errors. Breakpoint = stop at a chosen point.